



SHASU VATHANAN

SHASU VATHANAN - GEN AI - PRODUCT MANAGER

GEN AI ARCHITECT JOURNEY · PHASE-004

FastAPI Calculator API

Build, validate, and test a real JSON API in Python

Document	A practical build guide with the complete source
Level	Beginner to Intermediate
Tools	Python 3.9+ · FastAPI · Pydantic · Uvicorn
Source file	calculator.py
Phase	Phase-004 - Interfaces and APIs
Folder	Phase-004-FastAPI-And-Streamlit

Shasu Vathanan - GEN AI - Product Manager

SHASUVATHANAN.COM · [linkedin.com/in/shasuvathanan](https://www.linkedin.com/in/shasuvathanan)



Contents

This guide builds one small API end to end. It explains what an API is, sets up the environment, walks through the complete **calculator.py** source, then shows how to run and test every endpoint.

1	What you will build
2	Why FastAPI
3	Set up the environment
4	The complete application
5	How the code works
6	Run the API
7	The endpoints
8	Test it - Swagger UI, curl, and Postman
9	Error handling and status codes
10	Common problems and how to fix them

Before you start

You need Python 3.9 or newer and **pip**. No database, API key, or environment variable is required. Every command in this guide works in Windows PowerShell, Windows Command Prompt, macOS, and Linux - alternatives are shown where they differ.

1. What you will build

A calculator that runs as a web API instead of a terminal program.

Rather than typing numbers into a console, a client sends a JSON request over HTTP and receives a JSON response. This is exactly how production services expose functionality to web apps, mobile apps, automation scripts, and AI agents.

The finished API supports four operations, validates every incoming request, rejects division by zero with a clear error, reports its own health, and documents itself automatically.

Capability	How it is delivered
Four operations	add, subtract, multiply, divide
Typed request and response	Pydantic models - invalid data never reaches your logic
Meaningful errors	HTTP 400 for division by zero and unknown operations
Health endpoint	GET /health for uptime checks and monitoring
Interactive documentation	Swagger UI generated from the code itself



2. Why FastAPI

FastAPI turns ordinary Python functions into HTTP endpoints. You write a function, add a decorator above it, and the framework handles routing, JSON parsing, validation, serialization, status codes, and documentation.

What you write	What FastAPI does for you
A Python function	Exposes it at a URL and an HTTP method
A Pydantic model	Validates the request body and returns 422 on bad input
A return value	Serializes it to JSON with the correct content type
Type hints and docstrings	Builds the OpenAPI schema and the interactive docs

The key idea

The documentation is not written separately - it is generated from your code. Clear model names, type hints, and return values therefore produce clear documentation for free.

3. Set up the environment

Step 1 - Create a project folder

```
CMD
mkdir fastapi-calculator
cd fastapi-calculator
```

Step 2 - Create a virtual environment

```
CMD
python -m venv venv
```

On systems where Python is exposed as **python3**, use **python3 -m venv venv** instead.

Step 3 - Activate it

```
SHELL
# Windows PowerShell
venv\Scripts\Activate.ps1

# Windows Command Prompt
venv\Scripts\activate.bat

# macOS / Linux
source venv/bin/activate
```

When the environment is active, the prompt begins with **(venv)**.

Step 4 - Install FastAPI and the server

```
CMD
pip install fastapi uvicorn
```



FastAPI is the framework that defines the endpoints. **Uvicorn** is the server that actually receives HTTP requests and hands them to FastAPI. You need both.

4. The complete application

Save the following as **calculator.py**. This is the full, working source - nothing is omitted.

PYTHON

```
from fastapi import FastAPI, HTTPException
from fastapi.responses import HTMLResponse
from pydantic import BaseModel

# =====
# FastAPI Application
# =====

app = FastAPI(
    title="Calculator API",
    description="A simple calculator API using FastAPI",
    version="1.0.0"
)

# =====
# Request Model
# =====

class CalculationRequest(BaseModel):
    num1: float
    num2: float
    operation: str

# =====
# Response Model
# =====

class CalculationResponse(BaseModel):
    num1: float
    num2: float
    operation: str
    result: float
    message: str = "Calculation successful"
```

PYTHON

```
# =====
# Routes
# =====

@app.get("/", response_class=HTMLResponse)
def home():
    return """
    <h1>Calculator API is Running!</h1>
    """
```



```
<p>
    Go to <a href="/docs">/docs</a>
    to test the calculator interactively.
</p>
"""
```

```
@app.post("/calculate", response_model=CalculationResponse)
def calculate(data: CalculationRequest):
```

```
    num1 = data.num1
    num2 = data.num2
    op = data.operation.lower()

    # ADD
    if op == "add":
        result = num1 + num2
        message = f"{num1} + {num2} = {result}"

    # SUBTRACT
    elif op == "subtract":
        result = num1 - num2
        message = f"{num1} - {num2} = {result}"

    # MULTIPLY
    elif op == "multiply":
        result = num1 * num2
        message = f"{num1} x {num2} = {result}"
```

PYTHON

```
# DIVIDE
elif op == "divide":

    if num2 == 0:
        raise HTTPException(
            status_code=400,
            detail="Cannot divide by zero"
        )

    result = num1 / num2
    message = f"{num1} ÷ {num2} = {result}"

# INVALID OPERATION
else:
    raise HTTPException(
        status_code=400,
        detail="Invalid operation. Use: add, subtract, multiply, or divide."
    )

return CalculationResponse(
    num1=num1,
    num2=num2,
    operation=op,
    result=result,
    message=message
)
```



```
# =====  
# Health Check  
# =====  
  
@app.get("/health")  
def health_check():  
    return {  
        "status": "healthy",  
        "service": "Calculator API"  
    }
```

5. How the code works

The application object

FastAPI(title=..., description=..., version=...) creates the application. Those three values are not decoration - they appear at the top of the generated documentation and in the OpenAPI schema that other tools consume.

The request model

CalculationRequest declares that every request to **/calculate** must contain **num1**, **num2**, and **operation**, with the first two being numbers. FastAPI enforces this before your function runs. A request missing a field, or sending text where a number belongs, is rejected automatically with **422 Unprocessable Entity** and a precise explanation of which field failed.

The response model

CalculationResponse guarantees the shape of what goes back out. It echoes the inputs, adds the numeric **result**, and includes a human-readable **message**. Echoing the inputs is a small but valuable habit: a client reading a logged response can see exactly what produced it.

The routes

Each decorator binds an HTTP method and a path to a plain Python function. **GET /** returns an HTML landing page rather than JSON, which is why it declares `response_class=HTMLResponse`. **POST /calculate** does the work. **GET /health** returns a fixed dictionary that monitoring tools can poll.

The decision logic

`data.operation.lower()` normalises the operation so **"ADD"**, **"Add"**, and **"add"** all work. The **if/elif** chain then applies the matching arithmetic. Division is guarded before it executes, and any unrecognised operation falls through to the final **else**, so no request can ever reach the return statement without a valid result.

The request flow, end to end

Every call to **POST /calculate** passes through the same stages, and the order explains the status codes - 422 comes from validation, 400 from your own checks, so a badly typed request never reaches the arithmetic at all.

```
FLOW  
Client -> Uvicorn -> FastAPI route -> Pydantic validation  
                                     |  
                                     v  
JSON response <- CalculationResponse <- calculation logic
```



Why raise instead of return

`raise HTTPException(status_code=400, detail=...)` stops the function immediately and sends a proper HTTP error. Returning an error dictionary instead would send **200 OK** with a failure inside it - clients would treat the call as successful. Always signal failure with the status code.

6. Run the API

CMD
`uvicorn calculator:app --reload`

Part	Meaning
calculator	The Python file calculator.py
app	The FastAPI object created inside that file
--reload	Restarts the server after every code change (development only)

The server starts at **http://127.0.0.1:8000**. Keep the terminal open while using the API, and press **Ctrl + C** to stop it. When you are finished, leave the virtual environment with **deactivate**.

Development only

--reload watches your files and restarts on every save. That is ideal locally and wrong in production, where you would run Uvicorn without it, behind a process manager.

7. The endpoints

Method	Path	Purpose	Returns
GET	/	Landing page with a link to the docs	HTML
POST	/calculate	Performs one calculation	JSON (CalculationResponse)
GET	/health	Reports that the service is running	JSON
GET	/docs	Interactive Swagger UI	HTML
GET	/redoc	Alternative documentation view	HTML
GET	/openapi.json	The raw OpenAPI schema	JSON

A complete request and response

Send this body to **POST /calculate**:

JSON

```
{
  "num1": 25,
  "num2": 5,
  "operation": "divide"
}
```

The API replies with **200 OK** and:



JSON

```
{
  "num1": 25.0,
  "num2": 5.0,
  "operation": "divide",
  "result": 5.0,
  "message": "25.0 ÷ 5.0 = 5.0"
}
```

Note that **25** comes back as **25.0**. Pydantic converted the integer to the declared **float** type - the model is the contract, and the response is coerced to match it.

8. Test it - Swagger UI, curl, and Postman

Swagger UI - the fastest way

1. Open <http://127.0.0.1:8000/docs> in a browser.
2. Expand **POST /calculate**.
3. Select **Try it out**.
4. Edit the example body with your own numbers and operation.
5. Select **Execute** and read the response body and status code.

Swagger UI is generated from the Pydantic models, so the example body is always correct for the current version of your code.

curl

CMD

```
curl http://127.0.0.1:8000/health

curl -X POST http://127.0.0.1:8000/calculate ^
-H "Content-Type: application/json" ^
-d '{"num1":10,"num2":5,"operation":"add"}'
```

The **^** line continuations above are for Windows Command Prompt. On macOS, Linux, or PowerShell use a backslash and single quotes:

BASH

```
curl -X POST http://127.0.0.1:8000/calculate \
-H "Content-Type: application/json" \
-d '{"num1":10,"num2":5,"operation":"add"}'
```

Postman

1. Create a new request and set the method to **POST**.
2. Enter <http://127.0.0.1:8000/calculate> as the URL.
3. Open **Body**, choose **raw**, and select **JSON** from the content-type menu.
4. Paste a request body and select **Send**.
5. Confirm the status is **200 OK** and the **result** field is correct.

Postman sets **Content-Type: application/json** automatically when raw JSON is selected. No authorization is needed.



A test matrix worth running

Case	operation	num1, num2	Expected
Addition	add	10, 5	200 - result 15
Subtraction	subtract	10, 5	200 - result 5
Multiplication	multiply	10, 5	200 - result 50
Division	divide	10, 5	200 - result 2
Case-insensitive	ADD	10, 5	200 - result 15
Divide by zero	divide	10, 0	400 - Cannot divide by zero
Unknown operation	power	10, 5	400 - Invalid operation
Wrong type	add	"ten", 5	422 - validation error
Missing field	add	10 only	422 - validation error

9. Error handling and status codes

Status	When it happens	Who produced it
200 OK	A valid calculation completed	Your function
400 Bad Request	Division by zero, or an unknown operation	Your HTTPException
422 Unprocessable Entity	Missing field, wrong type, or malformed JSON	Pydantic, before your code runs
404 Not Found	The URL does not match any route	FastAPI
405 Method Not Allowed	A valid URL called with the wrong method, such as GET /calculate	FastAPI

A division-by-zero request returns this body alongside the 400 status:

```
JSON
{
  "detail": "Cannot divide by zero"
}
```

Two layers of protection

Pydantic guards the *shape* of the data - the right fields with the right types. Your **HTTPException** checks guard the *meaning* of the data - values that are well-formed but impossible to compute. A robust API needs both.

10. Common problems and how to fix them



Problem	Cause	Fix
uvicorn is not recognized	The environment is not active, or Uvicorn is installed elsewhere	Activate the venv, or run python -m uvicorn calculator:app --reload
Address already in use	Port 8000 is taken by another process	Run uvicorn calculator:app --reload --port 8001 and use the new port in your URLs
Error loading ASGI app	The file name or app variable in the command is wrong	The command is file:variable - check both halves match your file
405 on /calculate	It was opened in a browser address bar, which sends GET	Use Swagger UI, curl, or Postman - /calculate accepts POST only
422 on every request	The body is not raw JSON, or a field name is misspelt	Send raw JSON with exactly num1 , num2 , and operation
PowerShell blocks activation	Script execution policy restriction	Use venv\Scripts\activate.bat in Command Prompt, or review the execution policy before changing it

Next

Extend the API with a percentage or power operation, then add a request-history endpoint. Keep each addition small and test it before moving on.

Shasu Vathanan - GEN AI - Product Manager
SHASUVATHANAN.COM · linkedin.com/in/shasuvathanan